刘谦

求职岗位：中高级开发工程师（全栈偏前端）

核心能力：React / TypeScript / 复杂 ToB 系统 / 低代码 / Node.js 全栈与 AI 工程化

工作经验：6 年以上 | 电话：17317619053 | 邮箱：tunshisky@outlook.com

职业概述

6 年以上前端开发经验，长期参与智慧医疗、SaaS 数据平台和多端企业应用建设，以 React、TypeScript 和复杂 ToB 系统为核心方向。熟悉数据密集型页面、复杂表单、权限与流程、低代码配置、数据可视化及监控告警场景，能够独立推进需求澄清、技术方案、 开发联调、测试发布与线上问题排查。

前端是主要优势，同时具备 Node.js、NestJS、Prisma 和 MySQL 实战经验，能够完成数据建模、REST API、JWT 鉴权及前后端联调，独立打通“数据采集—接口服务—持久化与聚合—可视化—测试验证”的完整业务链路。

专业技能

React 与 TypeScript

1. 熟练使用 React Hooks、 React Router、 Zustand、 Redux Toolkit，能够根据状态归属和更新范围设计页面、模块及全局状态边界。
2. 熟悉 TanStack Query 的 queryKey 设计、缓存策略、请求失效、乐观更新和异常处理，能够区分服务端状态与客户端状态的管理边界。
3. 能够使用 TypeScript 设计接口模型、表单类型、组件 Props、联合类型及泛型工具，降低接口联调和业务重构中的类型风险。
4. 理解闭包、原型链、事件循环、 Promise、浏览器渲染机制和常见性能问题，能够结合实际现象定位重复渲染、状态旧值和异步时序问题。

ToB 业务、数据平台与工程化

1. 熟悉查询表格、复杂表单、数据字典、权限控制、流程状态、批量操作和多维筛选等企业级应用场景，能够处理字段密集、规则复杂的数据录入与管理页面。
2. 具备 ECharts 趋势图、柱状图、地图和关系图实践，能够设计数据转换层、统一图表配置、联动交互、自适应布局与空状态兜底。
3. 具备低代码配置面板、组件属性编辑、画布预览与运行时渲染实践，理解配置模型、组件注册、数据源和扩展边界设计。
4. 熟悉 Vite、Webpack、ESLint、Prettier 等工程工具，具备 Playwright E2E 测试和 Jenkins CI/CD 实践。

智能体协作与 AI 工程化

1. 采用目标与验收驱动的 AI 研发方式，先明确业务目标、功能边界、接口契约和完成标准，再让智能体参与方案分析与实现，关键规则和技术决策由本人负责。
2. 能够把完整需求拆分为产品分析、前端、后端、数据库、测试和评审等智能体任务，统一公共契约、任务依赖、修改范围和阶段验收，支持复杂功能分阶段交付。
3. 通过 AGENTS.md、CLAUDE.md、Rules、官方文档和既有代码组织稳定上下文，并使用 MCP、Skills 与 Plugins 接入需求、设计、接口及测试环境，为智能体提供受控工具与可信输入。
4. 采用“小步实现—真实联调—问题反馈—增量修正”的迭代方式，先验证核心链路，再逐步补充异常状态、权限边界、响应式和工程质量， 避免一次性生成大范围不可控修改。
5. AI 生成代码必须经过人工 Diff 审查、类型检查、自动化测试和生产构建；将评审与测试发现的高频问题回写到规则和上下文中，形成需求、实现、验证与知识沉淀闭环。

Node.js / NestJS 全栈能力

1. 能使用 Node.js、NestJS、Prisma 和 MySQL 开发实际业务接口，按业务模块拆分 Controller、Service、DTO 和数据访问逻辑。
2. 能够设计 REST API、数据库模型及关联关系，完成 DTO 参数校验、分页筛选、统一响应、异常处理和 Swagger 接口文档。
3. 具备基于 JWT 的 Access Token、Refresh Token、HttpOnly Cookie 和接口守卫实践，能够处理登录注册、令牌刷新、退出及受保护接口访问。
4. 能够完成 React、TanStack Query 与 NestJS 真实接口联调，并使用 Vitest 对 JWT 鉴权、DTO、业务规则和权限边界进行测试。

跨端与团队协作

1. 具备微信小程序、Taro、uni-app、Cordova 和 Electron 项目经验，了解 WebView 通信、平台 API 适配、分包和多端差异处理。
2. 能与产品、UI、后端和测试协作完成需求澄清、接口约定、风险识别及上线验收。

工作经历

蓝想数科 | 前端开发（前后端协作与 AI 工程化）

2025.06—至今

角色定位：以智慧医疗核心模块交付为主，负责将复杂临床规则转化为稳定、可验证的业务流程，并参与接口契约、 监控数据和智能体研发规范建设。

前端技术：React、TypeScript、TanStack Query、Zustand、Ant Design、Playwright

后端技术：Node.js、NestJS、Prisma、MySQL、REST API、DTO 校验

AI 工程：Claude Code、Cursor、MCP、Skills、Rules、智能体任务拆分与代码审查

1. 独立推进预检分诊、急诊抢救、重症监护和麻醉随访等核心模块，从需求澄清、方案拆解、接口约定推进到测试、上线和问题跟踪，对完整交付结果负责。
2. 将医学字段、数据字典、动态联动和状态规则整理为字段清单、校验矩阵与提交转换契约，使产品、前端、后端和测试能够围绕同一规则协作，降低需求理解偏差。
3. 从重复业务中沉淀表单项、字典选择器、校验及异常反馈等公共能力，在保持医疗场景差异的同时提高同类模块的一致性和可维护性。
4. 推动 Playwright 核心流程回归、监控埋点和接口日志排查形成质量闭环，使高频迭代具备可验证、可定位的交付保障。
5. 将目录规范、业务边界、接口资料和验收条件组织为智能体上下文，由 AI 辅助重复性实现，本人负责医疗规则、异常边界、代码审查与最终验收。
6. 围绕表单提交、状态流转和异常排查，与后端共同明确 API 契约、字段映射、参数校验、错误码和监控事件结构，从页面交互延伸到接口服务与数据模型理解完整业务链路。

上海领鸟云 | 前端开发

2022.07—2025.06

角色定位：SaaS 数据平台与低代码模块核心开发，从业务页面交付逐步扩展到公共组件、可视化体系和配置平台建设。

主要技术：React、TypeScript、TanStack Query、Zustand、Ant Design、ECharts、Jenkins

1. 独立承担客户管理、数据分析、选址分析和产业图谱等模块的需求分析、技术方案与交付，在产品、后端及数据团队之间推进接口和交互方案落地。
2. 将反复出现的查询、表格、字典和图表需求提升为公共能力，推动团队从逐页开发转向基于统一模式组合业务页面。
3. 深度参与低代码工具建设，将页面结构、组件属性和数据关系抽象为配置模型，支撑编辑、保存、预览与运行时渲染的完整链路。
4. 建立多类型图表的数据转换和接入方式，并参与 Jenkins 构建与多环境部署流程，兼顾业务交付效率、可视化一致性和发布稳定性。

上海联蔚数科 | 前端开发

2019.09—2022.07

角色定位：企业多端应用开发，完成从 React 页面实现到跨页面状态、多端适配和完整业务模块交付的能力积累。

主要技术：React、TypeScript、Redux、React Router、Ant Design、微信小程序、Taro、Cordova

1. 负责微信小程序、H5 和 PC 管理后台的业务模块开发，从页面实现逐步承担接口联调、状态设计、平台适配和上线交付。
2. 使用 Redux 管理跨页面共享状态，结合持久化与异步流程处理多步骤业务中的数据一致性，建立对复杂前端状态边界的系统认识。
3. 将公共业务逻辑与小程序、H5 平台能力分层，集中处理授权、分享、分包和平台 API 差异，降低多端重复实现与维护成本。
4. 围绕移动端体验实施路由懒加载、资源策略、请求合并和分包优化，形成兼顾功能交付、平台差异和性能体验的多端开发能力。

项目经验

企业级数据智能与低代码分析平台 | 配置驱动的可视化体系

项目背景：随着企业数据分析场景持续扩展，传统“一项需求对应一套页面”的开发方式难以应对多数据源、多分析维度和频繁变化的展示需求。平台需要同时支撑业务人员的数据查询、空间分析、关系洞察和自助式页面搭建，并保证不同模块在交互与数据表达上的一致性。

架构设计与关键实践：

1. 将页面能力抽象为查询、数据展示、字典解析、交互操作和详情编辑等可组合单元，通过明确的输入输出契约控制复用边界，使平台能力能够被不同业务模块按需组装。
2. 建立可视化数据适配层，将后端差异化数据转换为统一图表模型，再生成 ECharts 配置，集中处理趋势、分类、地图及关系数据的联动、 缩放、自适应和异常兜底。
3. 采用配置驱动思路描述页面结构、组件属性和数据源关系，打通“属性编辑—配置保存—页面预览—运行时渲染”链路，并通过组件注册机制为后续扩展保留边界。
4. 使用 TanStack Query 管理服务端数据缓存和失效，通过 Vite 构建拆分与 Jenkins 流水线形成从开发、检查到多环境交付的工程闭环。

项目价值：将分散的数据页面开发提升为可复用的平台能力，使数据接入、图表表达和页面搭建拥有统一模型，体现复杂 ToB 系统中的前端架构与产品抽象能力。

急诊重症临床协同平台｜高可靠流程、可观测性与智能体工程

项目背景：急诊抢救具有时间敏感、角色协同密集和数据准确性要求高的特点，医护人员需要在有限时间内综合患者状态、生命体征和处置进度。系统将分散的诊疗数据组织成连续业务流程，并通过自动化验证、监控埋点和日志排查保障高频迭代下的稳定性。

架构设计与关键实践：

1. 建立面向页面侧的统一数据模型，将生命体征及诊疗数据按指标值、单位、采集时间和状态组织，集中处理异常标记、数据缺失和接口失败，避免不同页面重复解释同类数据。
2. 以状态驱动方式组织抢救阶段、处置节点和关键时间信息，使医护人员能够快速识别当前进度、待处理事项与异常状态，并保留关键操作的追溯线索。
3. 将复杂业务拆分为字段配置、字典数据、校验规则、动态联动和提交转换五个层次，以校验矩阵和接口契约连接需求、开发及测试，控制规则散落带来的维护风险。
4. 建立 Playwright 核心流程回归、TypeScript 类型检查、生产构建、监控埋点和接口日志组成的质量门禁，覆盖正常流程、编辑回显、异常反馈与状态流转。
5. 围绕页面访问、关键操作、表单提交和接口异常设计监控事件字段，与后端接口错误码和日志信息对齐，形成从用户操作、前端状态到接口异常的问题定位链路。
6. 将目录规范、业务边界、接口资料与验收条件组织为智能体上下文，拆分实现和评审任务；生成内容必须经过人工规则核对与自动化验证后才能进入交付链路。

项目价值：把复杂页面建设提升为数据模型、状态流程、前后端契约、质量体系与监控链路的协同设计，形成“规则结构化—分层实现—监控反馈—自动化验证—人工验收”的交付方式。

极速购 AI 智能客服系统

技术栈：Vue3 · Node.js · Express · LangChain.js · LangGraph · RAG · pgvector · DeepSeek API

主导系统架构设计与全链路研发，构建可横向扩展的 AI 应用基础架构。

1. 建立模型抽象层：createModel 工厂函数封装所有模型配置，利用 DeepSeek / 智谱 AI / 百炼对 OpenAI 协议的兼容性，实现国内模型零切换成本接入，推广到团队作为模型接入规范
2. 设计 RAG 知识库工程化方案：文档切分策略（结构化文档 chunkSize 500 / overlap 50）、向量维度与存储结构设计、检索参数调优（Top-K / 相似度阈值）、Prompt 防幻觉设计（知识库无相关内容时明确告知），形成可复用的 RAG 落地方案
3. 基于 LangGraph 设计多 Agent 编排架构：意图识别节点解耦路由逻辑，各处理节点职责单一（订单 Agent / RAG 节点 / 通用对话），answerSynthesizer 汇合结果，扩展新意图类型只需增加节点，对已有路径无侵入
4. 设计前后端 SSE 协议规范：type 字段分类事件（node / steps / sources / answer / done / error），前端按事件类型驱动状态更新，协议版本可向后兼容，作为团队 AI 流式接口的设计标准